

Ingeniería de Software 3- Actualidad Informática

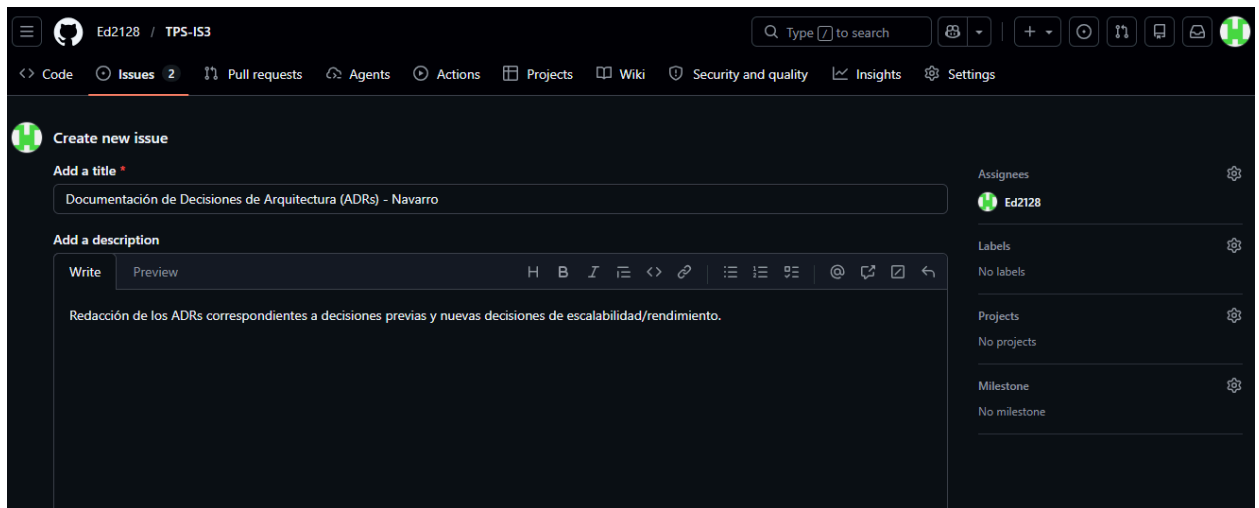
Trabajo Práctico Nro. 6 (2026) -Documentación de decisiones de arquitectura con ADRs

Alumnos:

- Navarro, Eduardo
- Proscopio Diaz, Karen
- Samudio, Paola
- Cotrina Wilches, Juan Diego

URL del repositorio remoto: [Ed2128/TPS-IS3: Repositorio de los trabajos prácticos de Ingeniería de Software 3, 2026.](#)

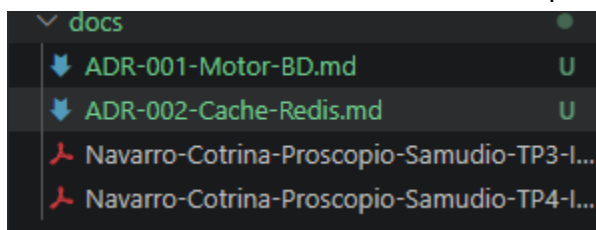
1. Crear Issue para la documentación de decisiones de arquitectura:



2. Creamos una nueva rama para trabajar.

```
PS C:\Users\edujn\Desktop\2026\Ing3\TPS-IS3> git checkout -b tp6-adrs-eduardo
Switched to a new branch 'tp6-adrs-eduardo'
PS C:\Users\edujn\Desktop\2026\Ing3\TPS-IS3>
```

3. Creamos los archivos Markdown en la carpeta /docs.



4. Completamos los ADRs siguiendo la plantilla.

ADR de decisión previa: Elección de PostgreSQL como motor de base de datos relacional.

```
ADR-001-Motor-BD.md X Preview ADR-001-Motor-BD.md ADR-002-Cache-Redis.md U
docs > ADR-001-Motor-BD.md > # Título: ADR-001 Elección de PostgreSQL como motor de base de datos relacional > ## Triggers de revisión
1 # Título: ADR-001 Elección de PostgreSQL como motor de base de datos relacional
2
3 **Estado:** Aceptado
4 **Fecha:** 2026-06-16
5 **Decisores:** Eduardo Navarro
6 **Relacionado:** Project.md (Stack Tecnológico Forzado)
7
8 ## Contexto
9 - **Qué problema se está resolviendo:** El Sistema de Gestión de Eventos Académicos (SGEA) requiere un sistema de almacenamiento persistente, robusto y transaccional para
manejar inscripciones, acreditaciones y certificados.
10 - **Qué restricciones aplican:** Se debe integrar nativamente con Java 21, Spring Boot 3.x y JPA/Hibernate.
11 - **Qué datos de proyecto sustentan la decisión:** El modelo de datos presenta alta relacionalidad entre Usuarios, Eventos e Inscripciones.
12
13 ## Decisión
14 - **Qué se decide exactamente:** Utilizar PostgreSQL como el motor de base de datos relacional principal.
15 - **Alcance:** Cubre el almacenamiento de todas las entidades de negocio. No cubre el almacenamiento de archivos binarios (los PDF de certificados se generan al vuelo o se
guardan en un Object Storage).
16
17 ## Alternativas consideradas
18 - **Opción A: MySQL / MariaDB**
19 - Pros: Amplia adopción, curva de aprendizaje baja.
20 - Contras: Menor cumplimiento estricto del estándar SQL en comparación con PostgreSQL, menor eficiencia en concurrencia masiva.
21 - **Opción B: MongoDB (NoSQL)**
22 - Pros: Esquema flexible, rápido desarrollo inicial.
23 - Contras: Ausencia de transacciones ACID nativas simples para relaciones complejas, lo que dificulta la lógica de acreditaciones e inscripciones limitadas por cupo.
24
25 ## Consecuencias
26 - **Beneficios esperados:** Consistencia transaccional fuerte, soporte avanzado para tipos de datos (como JSONB si a futuro se necesitan formularios de encuestas dinámicos), y
excelente compatibilidad con JPA.
27 - **Costos o riesgos que se aceptan:** La configuración de respaldos y monitoreo es ligeramente más compleja que en otras alternativas simples.
28 - **Impacto en operación y equipo:** El equipo ya configuró el entorno local con Docker en el TP5, por lo que el impacto operativo actual es nulo.
29
30 ## Plan de implementación
31 - **Pasos mínimos para ejecutarla:** Incluir la imagen 'postgres:15-alpine' en el 'docker-compose.yml' local. Configurar las variables 'SPRING_DATASOURCE_URL' en el backend.
32 - **Dependencias:** Driver de PostgreSQL en el 'pom.xml'.
33 - **Métrica de éxito:** Tiempos de respuesta de consultas complejas (ej. reporte de informes) por debajo de 500ms en desarrollo.
34
35 ## Triggers de revisión
36 - **Qué condiciones obligan a reabrir esta ADR:** Si el volumen de lecturas escala drásticamente y la base de datos comienza a presentar bloqueos o latencia superior a 2
segundos.
37 - **Fecha sugerida de revisión:** 2026-12-01.
```

ADR para la decisión ante un problema nuevo: Implementación de Caché con Redis para el listado público de eventos.

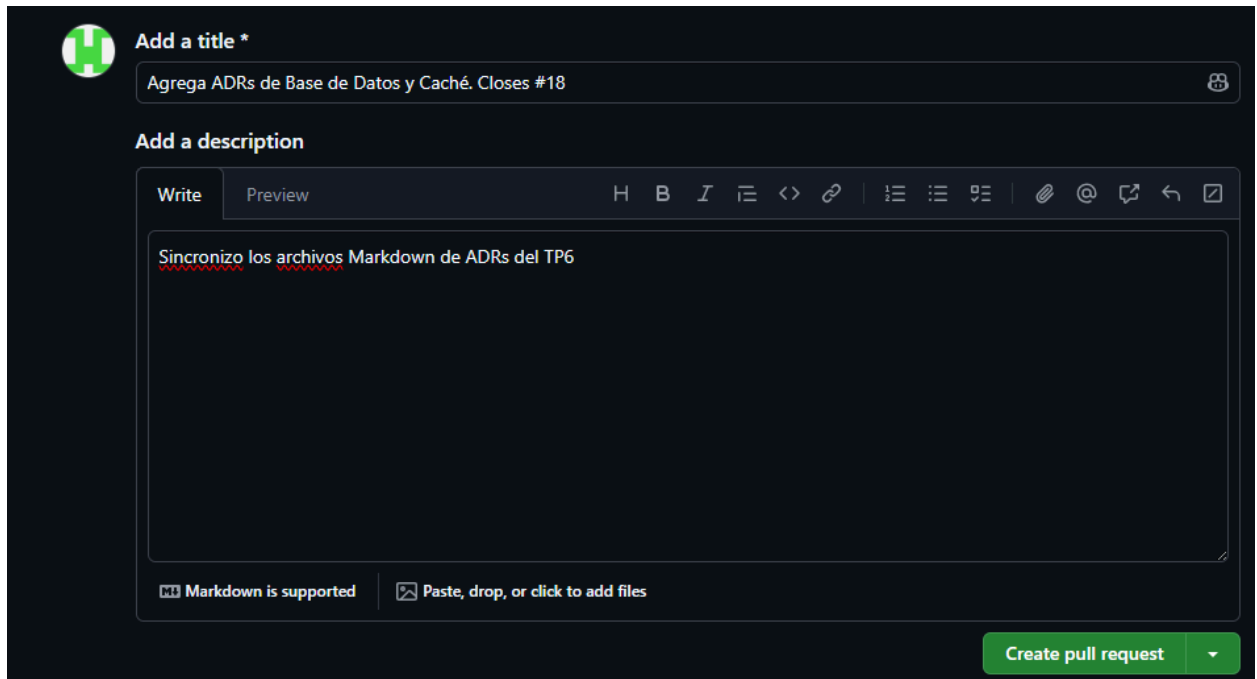
```
ADR-001-Motor-BD.md U ADR-002-Cache-Redis.md U X Preview ADR-002-Cache-Redis.md
docs > ADR-002-Cache-Redis.md > # Título: ADR-002 Implementación de Caché con Redis para el listado público de eventos > ## Triggers de revisión
1 # Título: ADR-002 Implementación de Caché con Redis para el listado público de eventos
2
3 **Estado:** Propuesto
4 **Fecha:** 2026-06-16
5 **Decisores:** Eduardo Navarro
6 **Relacionado:** Spec Gestión de Eventos
7
8 ## Contexto
9 - **Qué problema se está resolviendo:** Se anticipa una latencia alta en las consultas de la vista pública de próximos eventos ("Filtro de eventos a futuro"), debido a los
picos de tráfico durante las fechas de apertura de inscripciones.
10 - **Qué restricciones aplican:** La información debe servirse rápido para la UI en Next.js, pero no requiere estar actualizada al milisegundo absoluto.
11 - **Qué datos de proyecto sustentan la decisión:** La consulta a la tabla 'eventos' con filtros de fechas consume recursos de CPU en PostgreSQL de forma innecesaria cuando los
datos cambian poco frecuentemente.
12
13 ## Decisión
14 - **Qué se decide exactamente:** Introducir un sistema de caché en memoria utilizando Redis para almacenar los resultados del listado público de eventos.
15 - **Alcance:** Aplica únicamente al endpoint público de visualización de eventos. No aplica a módulos transaccionales (acreditaciones, asignación de roles o inscripciones).
16
17 ## Alternativas consideradas
18 - **Opción A: Caché local en memoria de Spring Boot (ConcurrentHashMap)**
19 - Pros: Fácil implementación sin infraestructura adicional.
20 - Contras: Si la aplicación escala a múltiples instancias, la caché se desincroniza.
21 - **Opción B: Escalar verticalmente PostgreSQL**
22 - Pros: No requiere cambiar el código del backend.
23 - Contras: Altamente costoso y no resuelve el problema de raíz de procesar peticiones idénticas repetidamente.
24
25 ## Consecuencias
26 - **Beneficios esperados:** Reducción del tiempo de respuesta del listado de eventos a < 50ms, aliviando la carga de PostgreSQL.
27 - **Costos o riesgos que se aceptan:** Se introduce una nueva pieza de infraestructura que administrar.
28 - Riesgo de "Stale Data" (mostrar eventos desactualizados por unos minutos).
29 - **Impacto en operación y equipo:** Los desarrolladores deberán aprender el uso de las anotaciones '@Cacheable' y '@CacheEvict' en Spring Boot.
30
31 ## Plan de implementación
32 - **Pasos mínimos para ejecutarla:** Añadir al contenedor de Redis al 'docker-compose.yml'. Agregar la dependencia 'spring-boot-starter-data-redis' en el proyecto. Intervenir el
'EventoService' con las anotaciones correspondientes.
33 - **Dependencias:** Servicio de Redis operativo.
34 - **Métrica de éxito:** El 90% de las peticiones públicas de eventos no impactan directamente contra la base de datos.
35
36 ## Triggers de revisión
37 - **Qué condiciones obligan a reabrir esta ADR:** Si el consumo de memoria RAM de Redis supera los límites asignados del servidor o si la invalidación de la caché genera
demasiados errores en la UI[cite: 372].
38 - **Fecha sugerida de revisión:** 2026-11-15.
```

5. Sincronizamos los cambios con el repositorio remoto y cerramos el issue:

```
PS C:\Users\edujn\Desktop\2026\Ing3\TPS-IS3> git add .
PS C:\Users\edujn\Desktop\2026\Ing3\TPS-IS3> git status
On branch tp6-adrs-eduardo
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   docs/ADR-001-Motor-BD.md
    new file:   docs/ADR-002-Cache-Redis.md

PS C:\Users\edujn\Desktop\2026\Ing3\TPS-IS3> git commit -m "Agrega ADRs de Base de Datos y Caché. Closes #18"
[tp6-adrs-eduardo 0a3978d] Agrega ADRs de Base de Datos y Caché. Closes #18
 2 files changed, 75 insertions(+)
 create mode 100644 docs/ADR-001-Motor-BD.md
 create mode 100644 docs/ADR-002-Cache-Redis.md
PS C:\Users\edujn\Desktop\2026\Ing3\TPS-IS3> git push origin tp6-adrs-eduardo
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 8 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 3.26 KiB | 556.00 KiB/s, done.
Total 5 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
remote: Create a pull request for 'tp6-adrs-eduardo' on GitHub by visiting:
remote:   https://github.com/Ed2128/TPS-IS3/pull/new/tp6-adrs-eduardo
remote:
To https://github.com/Ed2128/TPS-IS3.git
 * [new branch]      tp6-adrs-eduardo -> tp6-adrs-eduardo
```

6. Creamos una pull request:



The screenshot shows the GitHub interface for creating a pull request. At the top, there's a header with the repository name 'Ed2128/TPS-IS3' and a green 'H' icon. Below this, the title 'Add a title *' is followed by the text 'Agrega ADRs de Base de Datos y Caché. Closes #18'. The 'Add a description' section has a 'Write' tab selected, showing a rich text editor with the text 'Sincronizo los archivos Markdown de ADRs del TP6'. The editor includes a toolbar with various formatting options. At the bottom right, there is a green button labeled 'Create pull request'.